

函数设计文档

函数英文名	evaluatePolynomial	函数中文名	多项式求值
函数类型	元件 (Element)	版本	2.0
作者	许庆	日期	2026-06-26
联系方式	18612426814 / qingxu@tsinghua.edu.cn		
密级	内部公开	AI 辅助	Cursor Agent (Claude Opus 4.6)

更改记录				
版本	日期	更改说明	作者	辅助 AI
2.0	2026-06-26	初始版本	许庆	Claude Opus 4.6

目录

1	函数需求	3
1.1	功能概述	3
1.2	输入/输出/返回值	3
1.3	算法设计	3
1.4	程序流程图	4
1.5	函数接口	4
1.6	输入输出模块图	4
1.7	函数诸元表	4
1.8	函数架构	5
1.9	数据结构定义	5
1.9.1	PolynomialCoeffs — 三次多项式系数	5
1.10	测试用例	5
1.10.1	正常测试用例	5
1.10.2	异常测试用例	5

1 函数需求

1.1 功能概述

`evaluatePolynomial` 使用 Horner 法（秦九韶算法）计算三次多项式在给定点 x 处的值：

$$y = c_0 + c_1x + c_2x^2 + c_3x^3$$

Horner 法将其改写为嵌套乘法形式以减少乘法运算次数并提高数值稳定性。

1.2 输入/输出/返回值

类别	名称	类型	说明
输入	<code>coeffs</code>	const PolynomialCoeffs&	多项式系数 $c[0..3]$
输入	<code>x</code>	double	自变量值
返回值	—	double	多项式在 x 处的函数值

1.3 算法设计

三次多项式的标准形式为：

$$y = c_0 + c_1x + c_2x^2 + c_3x^3 \quad (1)$$

直接展开需要 6 次乘法和 3 次加法。Horner 法将其改写为嵌套形式：

$$y = c_0 + x \cdot (c_1 + x \cdot (c_2 + x \cdot c_3)) \quad (2)$$

仅需 3 次乘法和 3 次加法，计算效率提升 50%。同时由于从高阶向低阶逐步累加，中间结果的量级递减，有利于减少浮点舍入误差。

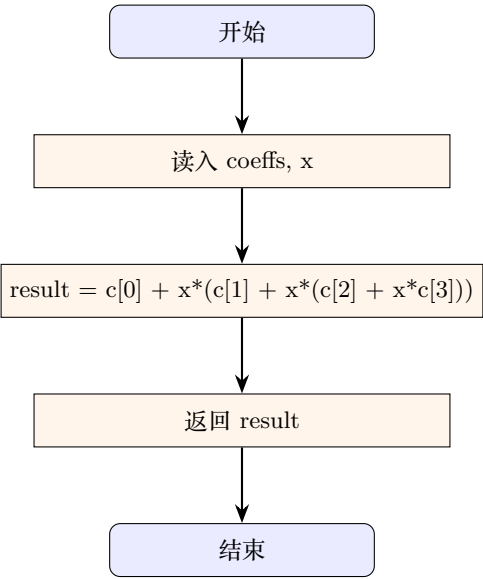
计算步骤：

1. $t_1 = c_2 + x \cdot c_3$
2. $t_2 = c_1 + x \cdot t_1$
3. $y = c_0 + x \cdot t_2$

实际实现中合并为单行表达式：

$$\text{result} = \text{coeffs.c}[0] + x \times (\text{coeffs.c}[1] + x \times (\text{coeffs.c}[2] + x \times \text{coeffs.c}[3]))$$

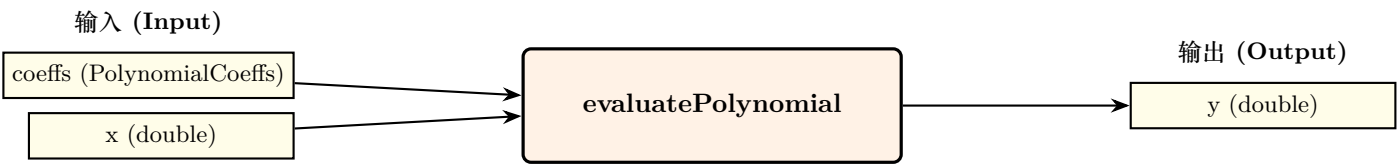
1.4 程序流程图



1.5 函数接口

```
double evaluatePolynomial(const PolynomialCoeffs& coeffs, const double x);
```

1.6 输入输出模块图



1.7 函数诸元表

属性	说明
函数英文名	evaluatePolynomial
函数中文名	多项式求值
函数类型	元件 (Element)
版本	2.0
源文件	src/cpp/evaluatePolynomial/evaluatePolynomial.cpp
头文件	include/cpp/evaluatePolynomial/evaluatePolynomial.hpp
命名空间	waypoint_follow
输入数量	2 (coeffs, x)
输出数量	1 (返回值 double)
参数数量	0
状态数量	0 (无状态, 纯函数)

属性	说明
下级函数数量	0
动态内存分配	无
外部依赖	无（仅算术运算）
算法类别	多项式计算 / Horner 法

1.8 函数架构

无下级函数调用。`evaluatePolynomial` 为叶节点元件，仅包含算术运算。

1.9 数据结构定义

引用 `waypointFollowTypes.hpp` 中定义的结构体：

1.9.1 PolynomialCoeffs — 三次多项式系数

成员	类型	说明
<code>c[4]</code>	<code>double[4]</code>	$y = c[0] + c[1]x + c[2]x^2 + c[3]x^3$

1.10 测试用例

1.10.1 正常测试用例

编号	场景	输入	预期输出	验证准则
N-1	零点求值	<code>c=[1, 2, 3, 4]</code> , $x=0$	$y = 1.0$	$y = c_0$ （高阶项消失）
N-2	$x=1$ 全系数	<code>c=[1, 2, 3, 4]</code> , $x=1$	$y = 10.0$	$1+2+3+4 = 10$
N-3	负 x 值	<code>c=[1, 2, 3, 4]</code> , $x=-1$	$y = -2.0$	$1-2+3-4 = -2$
N-4	仅常数项	<code>c=[5, 0, 0, 0]</code> , $x=100$	$y = 5.0$	零系数不影响结果
N-5	小数值 x	<code>c=[0, 1, 0, 0]</code> , $x=0.5$	$y = 0.5$	$y = c_1 \cdot x = 0.5$

正常测试用例符号说明	
符号	含义
<code>c</code>	多项式系数数组 <code>coeffs.c[0..3]</code>
x	自变量值
y	返回值（多项式函数值）
c_i	第 i 阶系数，即 <code>coeffs.c[i]</code>

1.10.2 异常测试用例

编号	场景	输入	预期输出/行为	验证准则
A-1	x 为 NaN	$\mathbf{c}=[1, 2, 3, 4]$, $x=\text{NaN}$	$y = \text{NaN}$	NaN 参与运算传播
A-2	x 为 $+\infty$	$\mathbf{c}=[1, 2, 3, 4]$, $x=+\infty$	$y = +\infty$	$c_3 > 0$ 时结果为 $+\infty$
A-3	x 为 $-\infty$	$\mathbf{c}=[1, 2, 3, 4]$, $x=-\infty$	$y = -\infty$	$c_3 > 0$, 奇数次幂主导
A-4	系数含 NaN	$\mathbf{c}=[1, \text{NaN}, 3, 4]$, $x=2$	$y = \text{NaN}$	NaN 系数传播
A-5	系数含 ∞	$\mathbf{c}=[+\infty, 0, 0, 0]$, $x=1$	$y = +\infty$	∞ 系数传播
A-6	$\infty - \infty$ 抵消	$\mathbf{c}=[+\infty, 0, 0, -\infty]$, $x=1$	$y = \text{NaN}$	$\infty + (-\infty) = \text{NaN}$
A-7	极大 x 值	$\mathbf{c}=[0, 0, 0, 1]$, $x=10^{100}$	$y = 10^{300}$	未溢出 ($< \text{DBL_MAX}$)
A-8	溢出场景	$\mathbf{c}=[0, 0, 0, 1]$, $x=10^{103}$	$y = +\infty$ (溢出)	$10^{309} > \text{DBL_MAX}$
A-9	全零系数	$\mathbf{c}=[0, 0, 0, 0]$, $x=10^{308}$	$y = 0.0$	精确零结果
A-10	subnormal x	$\mathbf{c}=[0, 1, 0, 0]$, $x=5 \times 10^{-324}$	$y \approx 5 \times 10^{-324}$	极小值正常处理

异常测试用例符号说明	
符号	含义
$\mathbf{c}$	多项式系数数组 <code>coeffs.c[0..3]</code>
x	自变量值
y	返回值 (多项式函数值)
NaN	IEEE 754 非数值, 任何含 NaN 的算术运算结果为 NaN
$+\infty/-\infty$	IEEE 754 正/负无穷大
DBL_MAX	double 最大有限值 $\approx 1.7 \times 10^{308}$
subnormal	非规格化浮点数 (极接近零)